

Guía de Ejercicios 6

Tecnología Digital VI

Juan Ignacio Elosegui

Universidad Torcuato Di Tella

Junio 2026

Respresentación de texto y embeddings

Ejercicio 1

La principal limitación de *Bag-of-Words* es que se olvida el orden de las palabras porque no es más que un histograma de las palabras que aparecen en un corpus. No solo eso, sino que si se aplica sobre un documento con muchas palabras, va a haber tantas columnas como palabras (altísima dimensionalidad); y también se ignora el contexto y la polisemia.

La normalización **TF-IDF** sirve para quitarle peso a las palabras que son muy frecuentes mediante dos mecanismos: *TF* e *IDF*.

- *TF*. Mide frecuencia de palabras en un documento.
- *IDF*. Penaliza palabras que aparecen en muchos documentos del corpus.

$$IDF = \log \left(\frac{N}{n_t} \right)$$

El producto $TF \times IDF$ le da más peso a palabras discriminativas.

¿Documento = corpus? No. Un corpus es una colección de documentos.

¿Qué son las palabras discriminativas? Son las palabras que ayudan a distinguir un documento del otro porque son específicas de ese documento.

¿Qué es n_t en el IDF? Es el número de documentos que contienen el término t . Y N es el número total de documentos del corpus.

Ejercicio 2

Con **OHE**, todos los vectores están a la misma distancia y son todos ortogonales entre sí, por lo que cualquier medida de similitud que se elija va a dar lo mismo para cualquier par de palabras. “Perro” va a ser igual de parecido a “Gato” que a “Chimenea”.

Word2Vec consiste de dos modelos que llevan palabras a un espacio latente donde las relaciones semánticas se preservan como operaciones vectoriales. Una palabra se caracteriza según qué palabras tenga alrededor.

- **CBOW**. Se predice la palabra del medio a partir de las n anteriores y las n posteriores.
- **Skip-Gram**. Es el proceso inverso: se predicen las n palabras alrededor de la central.

La diferencia trivial es que un modelo hace exactamente lo contrario al otro, pero la diferencia central es sobre qué palabras funciona mejor cada modelo.

Skip-Gram funciona mejor con palabras raras y corpus chicos porque genera más ejemplos de entrenamiento por palabra.

CBOW es más rápido de entrenar y anda bien con palabras frecuentes. Esto promedia el contexto y suaviza el ruido.

Preguntarle esto a Fede.

Ejercicio 3

La operación matemática sería

$$v_{\text{Madrid}} \approx v_{\text{Paris}} - v_{\text{Francia}} + v_{\text{España}}$$

Redes Neuronales Recurrentes (RNNs) y memoria

Ejercicio 4

El *hidden state* h_t es una retroalimentación del resultado anterior que, sumado con un input x_t (re)ingresan al bloque recurrente.

El hidden state se formula como

$$h_t = \sigma(W_1 x_t + W_2 h_{t-1})$$

(Con IA.) El hidden state es memoria porque en cada paso t se calcula a partir de h_{t-1} , que a su vez dependía h_{t-2} , y así sucesivamente: por recursión, h_t resume comprimido todo el contexto anterior de la secuencia en un vector de tamaño fijo. Eso es lo que le permite a la RNN “recordar” lo que vino antes al procesar la palabra actual.

En la práctica, la información de pasos muy lejanos se va diluyendo y el hidden state termina reflejando sobre todo el contexto reciente. Por eso se lo llama memoria a corto plazo: esa dilución es justamente el problema de vanishing gradients.

Ejercicio 5

Los gradientes dependen de la exponenciación de la matriz de pesos W . Multiplicar la misma matriz hace que los gradientes desaparezcan (como el vanishing gradient común) o que también exploten. Las RNNs no pueden aprender dependencias lejanas porque quiere decir que estamos hablando de más de treinta pasos para atrás.

Ejercicio 6

La **Long Short-Term Memory (LSTM)** soluciona el problema del vanishing gradient a la hora de aprender dependencias lejanas.

Contiene dos vectores de estado:

- **Hidden State** h_t . Es una memoria a corto plazo del contexto actual.
- **Cell State** c_t . Es la memoria a largo plazo que pasa información sin desvanecerse.

Tiene además tres vectores que controlan el flujo de la información:

- **Forget Gate**. Decide qué borrar del *cell state*. Se usa una sigmoide en la cual si devuelve cero, se debe olvidar; y si devuelve uno, se debe recordar. Puede haber valores intermedios?
- **Input Gate**. Decide cuánto de lo nuevo se escribe en el *cell state* (mediante una sigmoide) y qué cosa (mediante una tanh).
- **Output Gate**. Decide cuánto del *cell state* se expone como salida en h_t . Cómo sería esto?

Modelos Seq2Seq y Atención

Ejercicio 7 y 8

Seq2Seq toma secuencias de palabras para generar otras secuencias (por ejemplo, en traducción) usando dos LSTMs: un *encoder* que procesa la secuencia de entrada y genera una representación latente, y un *decoder* que genera la secuencia de salida usando la representación creada.

Attention es un mecanismo que permite al decoder mirar *todas* las posiciones de la secuencia de entrada, asignando pesos de importancia a cada posición según su relevancia para la palabra que se está generando. En criollo, el mecanismo sabe en qué palabra o parte de la frase enfocarse para dar la siguiente palabra en la traducción.

Si no existiera el mecanismo Attention, toda la información de la frase de entrada se comprime en un único vector resultante del encoder en el espacio latente. Puede llegar a funcionar para frases cortas pero no para párrafos completos. Este es el problema del **cuello de botella**, y Attention lo que hace para solucionarlo es permitirle ver al decoder todos los *hidden states* intermedios.

Preguntarle a Fede

Transformers y Large Language Models (LLMs)

Ejercicio 9

Se parte de una secuencia de palabras (w_i) que, antes que nada, se deben convertir a embeddings ($x_i = Ew_i$).

Luego, cada embedding se transforma en su (Query, Key, Value) = ($q_i = Qx_i, k_i = Kx_i, v_i = Vx_i$)

Se calculan similitudes e_{ij} : qué tan relacionada está cada palabra i con la palabra j . Se divide por la raíz de la dimensión de las queries, que es igual que la dimensión de las keys, en pos de normalizar la varianza y no sé qué más...

Se normalizan estas similitudes con una softmax, y la salida es una suma ponderada de los values.

“Es como una lookup table suavizada”, diría Fede.

Ejercicio 10

(Con IA). Se introdujo el **positional encoding**: a cada embedding se le suma un vector que codifica su posición dentro de la oración. Así, el vector que entra al modelo combina *qué* palabra es y *dónde* está.

Ejercicio 11

- BERT usa el encoder, GPT el decoder.
- BERT es bidireccional, GPT es únicamente de izquierda a derecha (unidireccional).
- BERT se usa para clasificación y análisis de reviews (si te gustó o no la película), GPT se usa para generación de texto.
- BERT está preentrenado con una arquitectura para predecir palabras ocultas y para predecir la siguiente oración. GPT se preentrenó para predecir la siguiente palabra.

Ejercicio 12

GPT antes respondía como si fuese un multiple choice en una prueba, lo que no era lo que esperaba el usuario.

InstructGPT resuelve esto con RLHF incorporando un modelo de recompensa con preferencias humanas. Basándose en aprendizaje por refuerzos (mirando su ambiente, lo que responde), se autoajusta para maximizar su recompensa.